

Assignment 1
COMP 325
University of the Fraser Valley
Instructor: Talia Q

By: Armaan Chima
Date: October 10th, 2025

The following assignment is given to help improve reverse-engineering skills, improve general understanding of Assembly language, and the process of constructing a plan and knowledge of extracting a password from the reverse-ex file using effective tools, methods, and strategies.

The assignment will include the plan and methods I used to approach this assignment, the challenges faced, lessons learned, a conclusion, and screenshots of my terminal and IDA (assembly breakdown).

The Steps I Took to Discover the Password

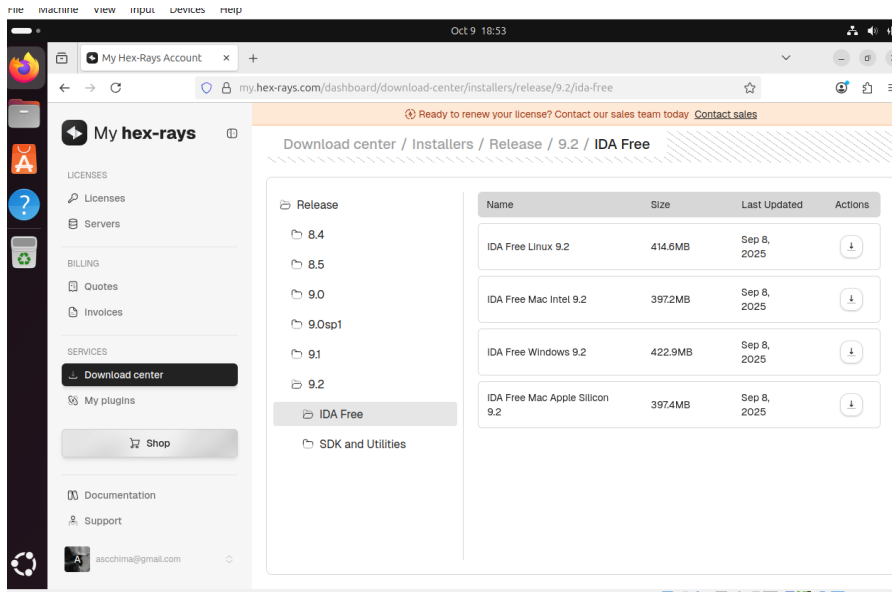
In order to discover what the password is from reverse-ex, I used tools such as IDA, GDB (debugger), and Ubuntu (VirtualBox). To approach this assignment, one must conduct this assignment in a controlled environment, where the following tools (IDA and GDB) will help unveil the password. This will be done through a mix of static analysis, dynamic analysis, and references (internal and external resources).

Methods Used

The first thing I did was install an Ubuntu server on VirtualBox by downloading an .iso image from the Ubuntu website. After that, I configured and installed GDB and IDA onto the Ubuntu server, so that the examination of the reverse-ex file could be conducted on a virtual machine instead of my own personal laptop environment. On the other hand, IDA will help convert the file to Assembly language. As for GDB, this will aid in debugging programs such as Java.

The following photos show the installation process of the tools that I will be using.

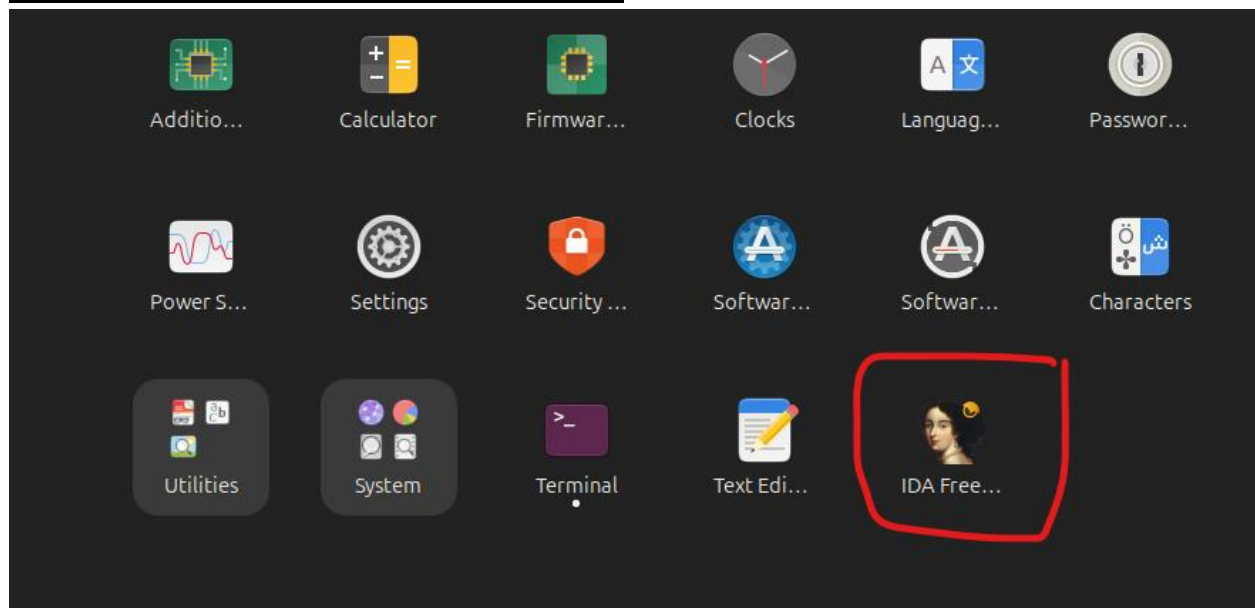
Installation of IDA on VM



Installation of GDB

```
ArmaanChima@COMP325Assignment1: ~/Downloads
Get:12 http://ca.archive.ubuntu.com/ubuntu questing-backports/multiverse amd64 C
omponents [216 B]
Get:13 http://security.ubuntu.com/ubuntu questing-security/main amd64 Components
 [208 B]
Get:14 http://security.ubuntu.com/ubuntu questing-security/restricted amd64 Comp
onents [212 B]
Get:15 http://security.ubuntu.com/ubuntu questing-security/universe amd64 Compon
ents [208 B]
Get:16 http://security.ubuntu.com/ubuntu questing-security/multiverse amd64 Comp
onents [212 B]
Fetched 107 kB in 1s (360 kB/s)
All packages are up to date.
ArmaanChima@COMP325Assignment1:~/Downloads$ sudo apt install -y gdb
gdb is already the newest version (16.3-1ubuntu2).
gdb set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
ArmaanChima@COMP325Assignment1:~/Downloads$ gdb --version
GNU gdb (Ubuntu 16.3-1ubuntu2) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
ArmaanChima@COMP325Assignment1:~/Downloads$
```

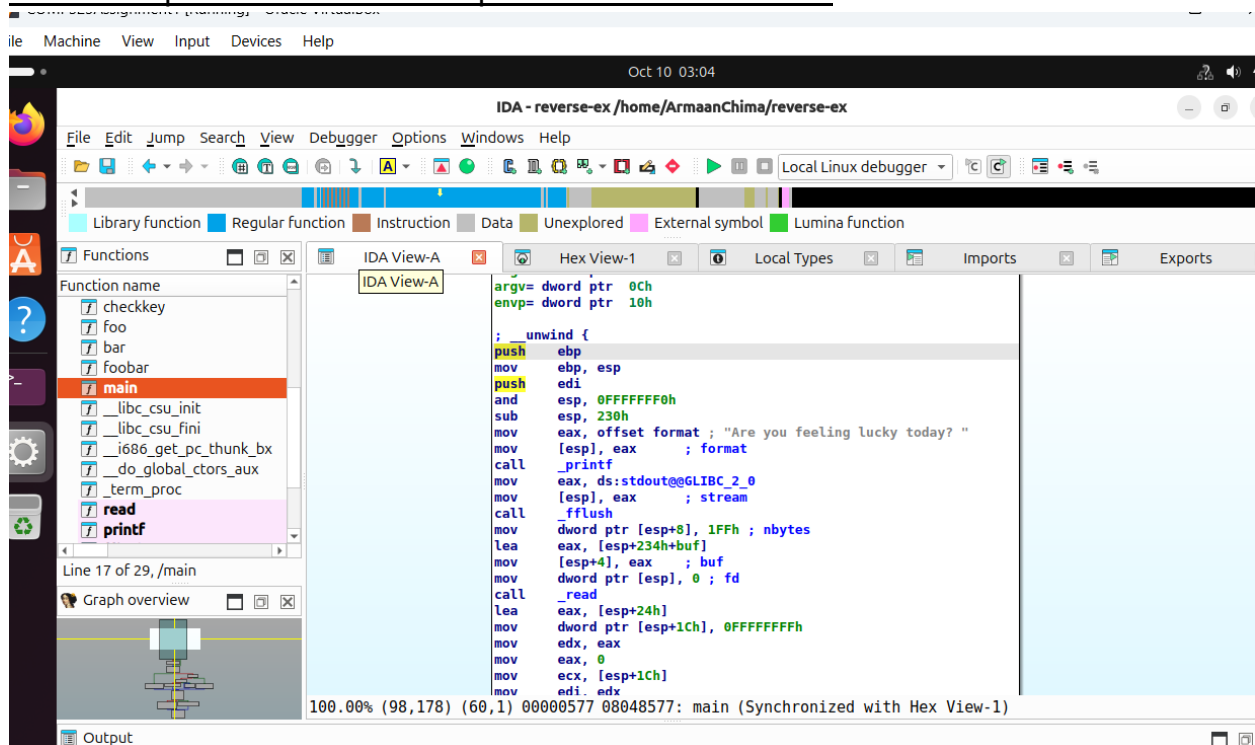
IDA is officially an application on the VM



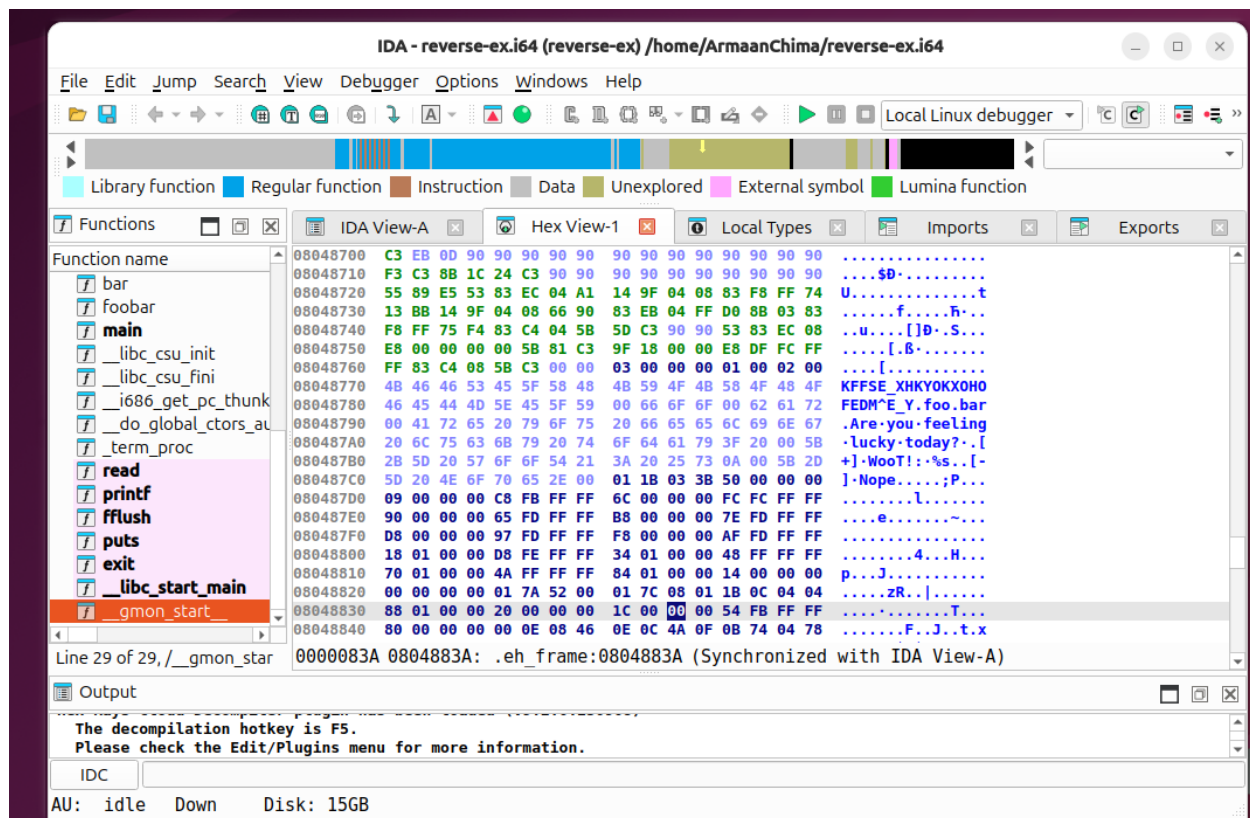
Once I had all my tools set up, installed and configured to my liking, I started running IDA and opened the reverse-ex file. From here, I got a better idea and understanding of what was in store for me and how I had to approach this code. Moreover, there were functions such as foo, bar, main, checkkey, and more, which aided in the discovery of this secret password. Functions like checkkey and main will offer clear insight into uncovering the hexadecimal key, such as looking for key instructions like eax, ebx or cmp, for instance.

In addition, the use of IDA-view and HEX-view will aid and offer different perspectives to help analyze the presented assembly code.

Below is a photo of what the sample looks like in IDA-view



Below is a photo of what the sample looks like in HEX-view



Moving on, since I can't execute the reverse-ex file due to having the wrong permissions implemented, I changed the permissions and was finally able to access the reverse-ex file. In the Ubuntu terminal, I typed in `./reverse-ex` and got the output "Are you feeling lucky today?". I am meant to enter a response to this prompt, but I do not have a response. However, after my first look at the assembly code in IDA, I did notice a commented message in the main function displaying, "Are you feeling lucky?" As a result, the main function should be inspected further to give better insight and hints to potentially find where the message originated from.

```

ArmaanChima@COMP325Assignment1: ~ — ./reverse-ex
ArmaanChima@COMP325Assignment1:~$ sudo chmod +x reverse-ex
[sudo: authenticate] Password:
ArmaanChima@COMP325Assignment1:~$ ls -l
total 424824
drwxr-xr-x  2 ArmaanChima ArmaanChima    4096 Oct  9 18:50 Desktop
drwxr-xr-x  2 ArmaanChima ArmaanChima    4096 Oct  9 18:50 Documents
drwxr-xr-x  2 ArmaanChima ArmaanChima    4096 Oct  9 23:08 Downloads
drwxr-xr-x  2 ArmaanChima ArmaanChima    4096 Oct  9 18:50 Music
drwxr-xr-x  2 ArmaanChima ArmaanChima    4096 Oct  9 18:50 Pictures
drwxr-xr-x  2 ArmaanChima ArmaanChima    4096 Oct  9 18:50 Public
drwxr-xr-x  2 ArmaanChima ArmaanChima    4096 Oct  9 18:50 Templates
drwxr-xr-x  2 ArmaanChima ArmaanChima    4096 Oct  9 18:50 Videos
drwxrwxr-x 14 ArmaanChima ArmaanChima    4096 Oct  9 22:40 ida-free-pc-9.2
-rwxrwxr-x  1 ArmaanChima ArmaanChima 434772719 Oct  9 19:02 ida-free-pc_92_x64linux.run
-rw-rw-r--  1 ArmaanChima ArmaanChima    1236 Oct  9 21:31 idafree_96-288F-F7A6-22.hexlic
-rwxrwxrwx  1 ArmaanChima ArmaanChima    7427 Oct  9 23:07 reverse-ex
-rw-rw-r--  1 ArmaanChima ArmaanChima   139264 Oct  9 23:09 reverse-ex.id0
-rw-rw-r--  1 ArmaanChima ArmaanChima   24576 Oct  9 23:09 reverse-ex.id1
-rw-rw-r--  1 ArmaanChima ArmaanChima     341 Oct  9 23:09 reverse-ex.id2
-rw-rw-r--  1 ArmaanChima ArmaanChima   16384 Oct  9 23:09 reverse-ex.nam
-rw-rw-r--  1 ArmaanChima ArmaanChima     540 Oct  9 23:09 reverse-ex.til
drwx----- 7 ArmaanChima ArmaanChima    4096 Oct 10 00:58 snap
ArmaanChima@COMP325Assignment1:~$ ./reverse-ex
Are you feeling lucky today?

```

Before touching on GDB, let's talk about the useful commands that will be needed! Below is a chart describing the useful commands for this assignment.

GDB Chart

Run	The program is being ran so it can be debugged
Break	This command sets a breakpoint
Continue	Normal execution continues as normal
Quit	Allows you to exit out of GDB
Step	This command does get into the functions and heads into the next instruction
Next	This command does not get into the functions, but rather just heads to the next instruction
x/s _____	Has the ability to look at memory at a certain address and display a string in return
info registers	Displays all values and names of all registers
Disassemble	This disassembles the function that is present

Now let's talk about using GDB with reverse-ex. I used the command `gdb ./reverse-ex`, which will essentially “put” reverse-ex into GDB. When this is executed, we will have the opportunity to work with the following commands listed in the above chart to allow us to

examine and uncover possible areas where data is being handled, getting us a step closer to cracking the secret passcode.

The image below shows me using breakpoints for the checkkey and main functions.

```
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
(No debugging symbols found in ./reverse-ex)
(gdb) break main
Breakpoint 1 at 0x804857b
(gdb) run
Starting program: /home/ArmaanChima/reverse-ex
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0804857b in main ()
(gdb) next
Single stepping until exit from function main,
which has no line number information.
Are you feeling lucky today? A
foo
[-] Nope.
[Inferior 1 (process 6928) exited normally]
(gdb) break checkkey
Breakpoint 2 at 0x80484c9
(gdb) run
Starting program: /home/ArmaanChima/reverse-ex
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0804857b in main ()
(gdb) next
Single stepping until exit from function main,
which has no line number information.
Are you feeling lucky today? A
foo
[-] Nope.
[Inferior 1 (process 6946) exited normally]
(gdb) step
The program is not being run.
(gdb) █
```

Commands such as break, run, and next are crucial since they allow you to analyze how the program is taking in input, and this can be seen when I type in A and get a foo and [-] Nope output (this is important later).

After this, I took the time to inspect the registers, hoping to find additional clues that could bring me a step closer to finding the password. Although this helped a bit with how

the program deals with input, I hit a roadblock and was unsure what to do from here.

```
>- Breakpoint 2 at 0x004857b
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/ArmaanChima/reverse-ex
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 2, 0x004857b in main ()
(gdb) info registers
eax            0xf7fa42b4            -134593868
ecx            0x9b12baab            -1693271381
edx            0xffffcf90            -12400
ebx            0xf7f9fe14            -134611436
esp            0xffffcf64            0xffffcf64
ebp            0xffffcf68            0xffffcf68
esi            0x0                    0
edi            0xf7ffcc60            -134230944
eip            0x004857b            0x004857b <main+4>
eflags         0x246                [ PF ZF IF ]
cs             0x23                35
ss             0x2b                43
ds             0x2b                43
es             0x2b                43
fs             0x0                    0
gs             0x63                99
(gdb) |
```

```
Starting program: /home/ArmaanChima/reverse-ex
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x004857b in main ()
(gdb) info registers
eax            0xf7fa42b4            -134593868
ecx            0xe707f17b            -418909829
edx            0xffffcf90            -12400
ebx            0xf7f9fe14            -134611436
esp            0xffffcf64            0xffffcf64
ebp            0xffffcf68            0xffffcf68
esi            0x0                    0
edi            0xf7ffcc60            -134230944
eip            0x004857b            0x004857b <main+4>
eflags         0x246                [ PF ZF IF ]
cs             0x23                35
ss             0x2b                43
ds             0x2b                43
es             0x2b                43
fs             0x0                    0
gs             0x63                99
(gdb) x/10x $eax
0xf7fa42b4: 0xffffd02c  0x00000000  0x00000000  0x00000000
0xf7fa42c4: 0x00000000  0x00000000  0x00000000  0x00000000
0xf7fa42d4: 0x00000000  0x00000000
(gdb) x/10x $edx
0xffffcf90: 0xf7f9fe14  0x00000000  0xf7ffcc60  0x00486a0
0xffffcfa0: 0xa867f76b  0xe707f17b  0x00000000  0x00000000
0xffffcfb0: 0x00000000  0xf7ffcc60
(gdb) x/10x $esp
0xffffcf64: 0xf7ffcc60  0x00486a0  0xf7d87fb5  0x00000001
0xffffcf74: 0xffffd024  0xffffd02c  0xffffcf90  0xf7f9fe14
0xffffcf84: 0x00000001  0xffffd024
(gdb) x/s 0x00486a0
0x00486a0 <__libc_csu_init>: "UWVS\350i"
(gdb) break checkkey
```

That is, until I did a bit of research and decided to disassemble the main function, which allowed me to find “\$0x8048791”, which translates to “Are you feeling lucky today?”

However, I needed to find foo, bar, and the [+] Woot!: %s/n"

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x08048577 <+0>:    push    %ebp
   0x08048578 <+1>:    mov     %esp,%ebp
   0x0804857a <+3>:    push    %edi
=>  0x0804857b <+4>:    and     $0xffffffff0,%esp
   0x0804857e <+7>:    sub     $0x230,%esp
   0x08048584 <+13>:   mov     0x08048791,%eax
   0x08048589 <+18>:   mov     %eax,(%esp)
   0x0804858c <+21>:   call    0x080483b0 <printf@plt>
   0x08048591 <+26>:   mov     0x0804a040,%eax
   0x08048596 <+31>:   mov     %eax,(%esp)
   0x08048599 <+34>:   call    0x080483c0 <fflush@plt>
   0x0804859e <+39>:   movl    $0x1ff,0x8(%esp)
   0x080485a6 <+47>:   lea     0x24(%esp),%eax
   0x080485aa <+51>:   mov     %eax,0x4(%esp)
   0x080485ae <+55>:   movl    $0x0,(%esp)
   0x080485b5 <+62>:   call    0x080483a0 <read@plt>
   0x080485ba <+67>:   lea     0x24(%esp),%eax
   0x080485be <+71>:   movl    $0xffffffff,0x1c(%esp)
   0x080485c6 <+79>:   mov     %eax,%edx
   0x080485c8 <+81>:   mov     $0x0,%eax
   0x080485cd <+86>:   mov     0x1c(%esp),%ecx
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
```

However, I needed to find foo, bar, and the [+] Woot!: %s\n". As a result of doing research online, I used the command x/64bx 0x08048780, which displays the 64-bit memory that begins at 0x08048780. This information is displayed in hexadecimal.

If we convert red, we get foo, green gets us bar, orange gets is Are you feeling lucky today?, and blue gets us [+] WooT!: %s\n.

```
(gdb) x/64bx 0x08048780
0x08048780: 0x46 0x45 0x44 0x4d 0x5e 0x45 0x5f 0x59
0x08048788: 0x00 0x66 0x6f 0x6f 0x00 0x62 0x61 0x72
0x08048790: 0x00 0x41 0x72 0x65 0x20 0x79 0x6f 0x75
0x08048798: 0x20 0x66 0x65 0x65 0x6c 0x69 0x6e 0x67
0x080487a0: 0x20 0x6c 0x75 0x63 0x6b 0x79 0x26 0x74
0x080487a8: 0x6f 0x64 0x61 0x79 0x3f 0x20 0x00 0x5b
0x080487b0: 0x2b 0x5d 0x20 0x57 0x6f 0x6f 0x54 0x21
0x080487b8: 0x3a 0x20 0x25 0x73 0x0a 0x00 0x5b 0x2a
(gdb)
```

So, from this information, I reset my GDB and inserted the following inputs, which gave me my desired output. See the image below for reference.

```
(No debugging symbols found in /lib/.../libc.so.6)
(gdb) x/s 0x08048789
0x08048789: "foo"
(gdb) x/s 0x0804878d
0x0804878d: "bar"
(gdb) x/s 0x080487af
0x080487af: "[+] Woot!: %s\n"
(gdb) x/s 0x08048791
0x08048791: "Are you feeling lucky today? "
(gdb)
```

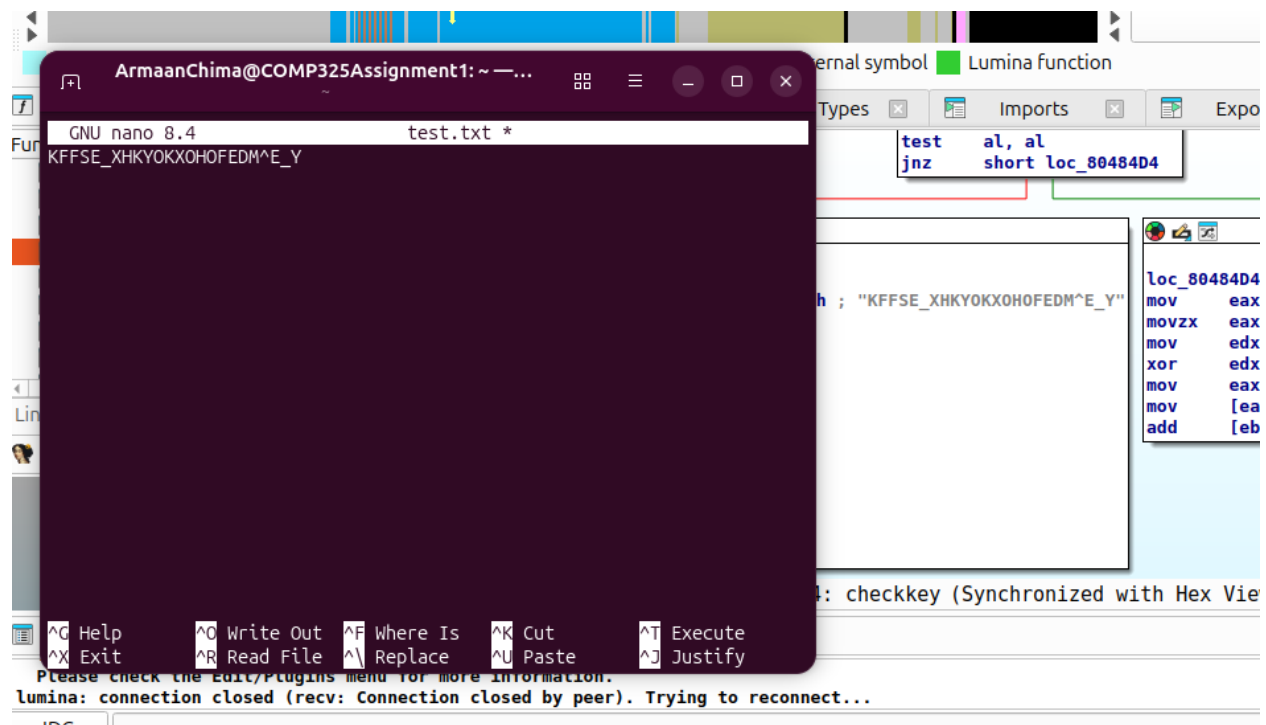
However, there is more. While further inspecting the checkkey function, I came across a mov instruction, which is moving information to edx from eax. After that, we can see that XOR and EDX are being differentiated with 2Ah, which must be the “stand-in” key in question.

```
loc_80484D4:
mov     eax, [ebp+var_C]
movzx   eax, byte ptr [eax]
mov     edx, eax
xor     edx, 2Ah
mov     eax, [ebp+var_C]
mov     [eax], dl
add     [ebp+var_C], 1
```

This is great, we are very close to cracking the passcode. Next in the checkkey function, I will need to find the XOR-encrypted string. After a bit of digging, I was able to find it.

```
mov     eax, [ebp+arg_0]
mov     edx, eax
mov     eax, offset aKffseXhkyokxoh ; "KFFSE_XHKYOKXOHOFEDM^E_Y"
mov     ecx, 15h
mov     esi, edx
mov     edi, eax
repe cmpsb
setnbe  dl
setb    al
mov     ecx, edx
sub     cl, al
mov     eax, ecx
movsx   eax, al
test    eax, eax
jnz     short loc_8048521
```

From here, I create a test.txt file where I put “KFFSE_XHKYOKXOHOFEDM^E_Y”



Next, I install Java through my terminal.

```
ArmaanChina@COMP325Assignment1:~$ sudo apt install openjdk-17-jdk
```

```
ArmaanChina@COMP325Assignment1:~$ javac -version
javac 17.0.17-ea
```

After installation, I created a .java file called decipher.java where I coded the following to decipher the XOR-encrypted string (KFFSE_XHKYOKXOHOFEDM^E_Y) and turn it into readable text.

```
//Personal note: took reference from lecture notes
public class decipher {
    public static void main(String[] args) {

        byte[] values = {
            0x4B, 0x46, 0x46, 0x53, 0x45, 0x5F, 0x58, 0x48, 0x4B, 0x59,
            0x4F, 0x4B, 0x58, 0x4F, 0x48, 0x4F, 0x46, 0x45, 0x44,
            0x4D, 0x5E, 0x45, 0x5F, 0x59, 0x00
        };

        byte key = 0x2A;
        String decipher = "";

        for (byte x : values) {
            if (x == 0)
                break;

            decipher += (char)(x ^ key);
        }

        System.out.println(decipher);
    }
}
```

The process of how I got the values in the array is very simple. I used a command called `xxd -p test.txt`, which took the XOR-encrypted string and turned it into hexadecimal. From there, I broke each value into pairs, which is how I got the values that are shown in the array.

```
ArmaanChina@COMP325Assignment1:~$ xxd -p test.txt
4b464653455f58484b594f4b584f484f4645444d5e455f590a
```

Now that everything is set, we can run the command `javac decipher.java` followed by the command `java decipher.java`, and we have our fully legible response “allyourbasearebelongtous”.

```
ArmaanChina@COMP325Assignment1:~$ javac decipher.java
ArmaanChina@COMP325Assignment1:~$ java decipher.java
allyourbasearebelongtous
ArmaanChina@COMP325Assignment1:~$
```

We have found the message “allyourbasearebelongtous”. However, this is not the complete passcode, since when this input is used, we get no response, indicating that there is still more work to be done.

```
ArmaanChina@COMP325Assignment1:~$ ./reverse-ex
Are you feeling lucky today? allyourbasearebelongtous
```

However, after further analyzing the checkkey function, I came across the following section of code!

```
.text:08048605 loc_8048605: ; CODE XREF: main+7D↑j
.text:08048605 movzx  eax, byte ptr [esp+24h]
.text:0804860A movsx  eax, al
.text:0804860D cmp    eax, 42h ; 'B'
.text:08048610 jz     short loc_8048629
.text:08048612 cmp    eax, 43h ; 'C'
.text:08048615 jz     short loc_8048636
.text:08048617 cmp    eax, 41h ; 'A'
.text:0804861A jnz     short loc_8048643
.text:0804861C mov    dword ptr [esp+22Ch], offset foo
.text:08048627 jmp    short loc_8048645
.text:08048629 ; -----
.text:08048629 loc_8048629: ; CODE XREF: main+99↑j
.text:08048629 mov    dword ptr [esp+22Ch], offset checkkey
.text:08048634 jmp    short loc_8048645
.text:08048636 ; -----
```

This section was the key to finding the last piece of the password. As a result, I went back into the terminal, typed “gdb ./reverse-ex” once again, and tested how each single letter reacted to the “Are you feeling lucky today” message. The findings can be shown below.

```
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
(No debugging symbols found in ./reverse-ex)
(gdb) break main
Breakpoint 1 at 0x0804857b
(gdb) run
Starting program: /home/ArmaanChina/reverse-ex
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0804857b in main ()
(gdb) next
Single stepping until exit from function main,
which has no line number information.
Are you feeling lucky today? A
foo
[.] Nope.
[Inferior 1 (process 6928) exited normally]
(gdb) break checkkey
Breakpoint 2 at 0x080484c9
(gdb) run
Starting program: /home/ArmaanChina/reverse-ex
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0804857b in main ()
(gdb) next
Single stepping until exit from function main,
which has no line number information.
Are you feeling lucky today? A
foo
[.] Nope.
[Inferior 1 (process 6946) exited normally]
(gdb) step
The program is not being run.
(gdb)
```

```
This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
(No debugging symbols found in ./reverse-ex)
(gdb) break main
Breakpoint 1 at 0x0804857b
(gdb) run
Starting program: /home/ArmaanChima/reverse-ex
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0804857b in main ()
(gdb) next
Single stepping until exit from function main,
which has no line number information.
Are you feeling lucky today? B
[-] Nope.
[Inferior 1 (process 7103) exited normally]
(gdb) break checkkey
Breakpoint 2 at 0x080484c9
(gdb) run
Starting program: /home/ArmaanChima/reverse-ex
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0804857b in main ()
(gdb) next
Single stepping until exit from function main,
which has no line number information.
Are you feeling lucky today? B

Breakpoint 2, 0x080484c9 in checkkey ()
(gdb) step
Single stepping until exit from function checkkey,
which has no line number information.
0x08048658 in main ()
(gdb)
```

```

his GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
enable debuginfod for this session? (y or [n]) y
debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
No debugging symbols found in ./reverse-ex
(gdb) break main
Breakpoint 1 at 0x804857b
(gdb) run
Starting program: /home/ArmaanChima/reverse-ex
Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0804857b in main ()
(gdb) next
Single stepping until exit from function main,
which has no line number information.
Are you feeling lucky today? C
foo
[-] Nope.
Inferior 1 (process 6986) exited normally]
(gdb) break checkkey
Breakpoint 2 at 0x80484c9
(gdb) run
Starting program: /home/ArmaanChima/reverse-ex
Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0804857b in main ()
(gdb) next
Single stepping until exit from function main,
which has no line number information.
Are you feeling lucky today? C
foo
[-] Nope.
Inferior 1 (process 6989) exited normally]
(gdb) step
The program is not being run.
(gdb)

```

Notice that when I type A and C, I get the output foo followed by a [-] Nope and when I type B, I get only [-] Nope.

So, what does this mean?

This means that `cmp eax, 41h` ; 'A' and `cmp eax, 43h` ; 'C' set off another function like foo, for example (which is incorrect). However, `cmp eax, 42h` ; 'B' sets off the checkkey function, which implies that B is a part of the passcode.

Below, we will see the final successful and correct passcode

```

ArmaanChima@COMP325Assignment1:~$ ./reverse-ex
Are you feeling lucky today? Ballyourbasearebelongtous
[+] Woot!: KFFSE_XHKYOKXOHOFEDM^E_Y

```

Challenges:

Some of the challenges that I faced were...

- Installing “IDA free” on the Ubuntu virtual machine

- Learning and understanding Assembly language. Since I have no prior experience with the language, it took a lot of time to understand basic syntax and principles.
- Getting a general grasp of the common GDB commands and their functions (understanding run, break, continue, etc.)
- Figuring out GDB, since it was my first time working with the debugger, it was overwhelming, and I sometimes had a difficult time comprehending the outputs. As a result, it took time to interpret the information that was presented to me.
- Analyzing the functions was a pretty tedious task since I had to go back many times to certain functions and cross-reference, which resulted in a lot of experimentation and testing

Lessons Learned:

Some of the lessons that I have learned are...

- Learning how to analyze Assembly code at a higher level and having the ability to pick up clues and hints from said code.
- IDA free turned out to be an excellent tool for this assignment, and now I know that it is a necessity for assignments like this in order to get a successful result.
- I learned a plethora of new terminal commands, more specifically for debugging
- Learning how to configure and install GDB, as well as learn how to operate it, was an exciting and informative experience
- I had the opportunity to crack this password, which taught me to stay disciplined, resilient and offered a new perspective on encryption and general security principles

Conclusion:

Cracking passwords is a challenging process that involves an immense amount of time, effort and a well-thought-out plan in order to succeed. With the help of IDA Free, I had the opportunity to thoroughly analyze the presented Assembly code and find a way to reveal the secret passcode. The main, checkkey, foo, and bar functions offer different perspectives and views of the code's formation and movement, which helped in my search. In addition, the hexadecimal key was revealed via XOR encryption, which also played another major part. Furthermore, looking at the main function, I was able to identify crucial features and clues to see how the program was reacting to my input, which allowed me to find locations of important code that played an important part in the process of finding the hidden password. Overall, this was a very interesting and fun project that really put decryption and reverse engineering skills to the test.

References:

ProgrammingKnowledge. (2024, May 15). *How to Install Ubuntu 24.04 LTS on VirtualBox in Windows 11 (2024)*. YouTube.

<https://www.youtube.com/watch?v=DhVjgl57Ino>

Hex-Rays. (2022, September 27). *How to download & install IDA Pro*. YouTube.

<https://www.youtube.com/watch?v=CXsN7uTN-nY>

Bohemian Digital. (2025, January 23). *XOR Encryption*. YouTube.

<https://www.youtube.com/watch?v=jxDGDh4ngSk>

Derek Banas. (2016, April 30). *Assembly Language Tutorial*. YouTube.

<https://www.youtube.com/watch?v=ViNnfoE56V8>

GeeksforGeeks. (2019, March 8). *gdb command in Linux with examples*.

GeeksforGeeks. <https://www.geeksforgeeks.org/linux-unix/gdb-command-in-linux-with-examples/>

The javac Command. (2021). Oracle.com.

<https://docs.oracle.com/en/java/javase/16/docs/specs/man/javac.html#compiling-for-earlier-releases-of-the-platform>